

# Final Project: BeautifulSoup Documentation Analytics System

**Team Project - Course: PDS301m (Python for Data Science)**

**Project Goal:** Build an end-to-end data pipeline to collect, parse, extract, and analyze structural metadata from the official BeautifulSoup documentation page (Feature 6 Focus).

**Target URL:** <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

## Project Architecture

The core logic lives in the `src/` Python package, allowing both scripts and interactive notebooks to run the same code:

| File                                 | Component                         | Description                                              |
|--------------------------------------|-----------------------------------|----------------------------------------------------------|
| <code>src/collector.py</code>        | Feature 1: Web Page Collector     | Downloads and saves raw HTML                             |
| <code>src/parser.py</code>           | Feature 2: HTML Parser            | Parses HTML with BeautifulSoup                           |
| <code>src/extractor.py</code>        | Features 3, 4, 5: Data Extractors | Extracts Sections, Links, and Code Examples to CSVs      |
| <code>src/analyzer.py</code>         | Feature 6: Doc Analytics          | Answers 8 required and 2 additional analytics questions  |
| <code>src/visualizer.py</code>       | Feature 7: Data Visualization     | Generates 4 charts saved to <code>output/charts/</code>  |
| <code>src/report_generator.py</code> | Feature 8: Final Report           | Generates Markdown & HTML report in <code>output/</code> |
| <code>src/main.py</code>             | Orchestration Pipeline            | Runs the entire pipeline sequentially (Features 1-8)     |

The cells below walk through the entire pipeline step-by-step.

## 1. Import Required Libraries

```
In [1]: import os
import sys
import re
import requests
import pandas as pd
import numpy as np
from bs4 import BeautifulSoup
```

```
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 100)
```

## 2. Set Project Working Directory & Paths

```
In [2]: # Set current directory to project root
if "src" in os.listdir(".") or "notebooks" in os.listdir("."):
    print("Currently in root.")
else:
    PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), os.pardir))
    os.chdir(PROJECT_ROOT)
    print("Changed working directory to project root:", os.getcwd())

sys.path.insert(0, os.getcwd())

# Create directories
os.makedirs("data/raw", exist_ok=True)
os.makedirs("data/processed", exist_ok=True)
print("Folders checked & initialized.")
```

Changed working directory to project root: D:\PRJ\FinalProjectPDS301m  
Folders checked & initialized.

## 3. Feature 1 – Web Page Collector

Downloads the official BeautifulSoup documentation using `requests` and saves it to `data/raw/beautifulsoup_doc.html`.

```
In [3]: from src import collector

html = collector.collect()
print("Feature 1 completed successfully.")
```

Downloading: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>  
Raw HTML saved to: data\raw\beautifulsoup\_doc.html (380,701 characters)  
Feature 1 completed successfully.

## 4. Feature 2 – HTML Parser

Parses the local HTML file into a BeautifulSoup tree structure.

```
In [4]: from src import parser

soup = parser.parse()
print("Feature 2 completed successfully.")
```

Loading HTML from: data\raw\beautifulsoup\_doc.html  
Page title: Beautiful Soup Documentation – Beautiful Soup 4.14.3 documentation  
Feature 2 completed successfully.

## 5. Feature 3 – Section Extractor

Extracts sections (h1, h2, h3 headings), computes section word count, code blocks count, and links count, saving results to `data/processed/sections.csv` .

```
In [5]: from src import extractor

sections_df = extractor.extract_sections(soup)
extractor.save_sections(sections_df)
print(f"Feature 3 completed: Extracted {len(sections_df)} sections.")
sections_df.head(5)
```

Feature 3 completed: Extracted 113 sections.

Out[5]:

|  | section_id | section_level | section_title | section_text | word_count | code_block_count |
|--|------------|---------------|---------------|--------------|------------|------------------|
|--|------------|---------------|---------------|--------------|------------|------------------|

|   |   |    |                                |                                                                                                     |     |    |
|---|---|----|--------------------------------|-----------------------------------------------------------------------------------------------------|-----|----|
| 0 | 1 | h1 | Beautiful Soup Documentation ¶ | Beautiful Soup is a Python library for pulling data out of HTML and XML files. It works with you... | 310 | 0  |
| 1 | 2 | h2 | Getting help ¶                 | If you have questions about Beautiful Soup, or run into problems, send mail to the discussion gr... | 97  | 0  |
| 2 | 3 | h3 | API documentation ¶            | This document is written like an instruction manual, but you can also read traditional API docum... | 43  | 0  |
| 3 | 4 | h1 | Quick Start ¶                  | Here's an HTML document I'll be using as an example throughout this document. It's part of a sto... | 422 | 6  |
| 4 | 5 | h1 | Installing Beautiful Soup ¶    | If you're using a recent version of Debian or Ubuntu Linux, you can install Beautiful Soup with ... | 431 | 18 |

## 6. Feature 4 – Link Extractor

Extracts all hyperlinks, classifies them based on location and destination rules, and saves them to `data/processed/links.csv`.

```
In [6]: links_df = extractor.extract_links(soup)
        extractor.save_links(links_df)
        print(f"Feature 4 completed: Extracted {len(links_df)} links.")
        links_df.head(5)
```

Feature 4 completed: Extracted 504 links.

```
Out[6]:
```

|   | link_text           | href                                                                |        |
|---|---------------------|---------------------------------------------------------------------|--------|
| 0 | ¶                   | #beautiful-soup-documentation                                       | inte   |
| 1 | Beautiful Soup      | http://www.crummy.com/software/BeautifulSoup/                       | docume |
| 2 | Beautiful Soup 3    | http://www.crummy.com/software/BeautifulSoup/bs3/documentation.html | docume |
| 3 | Porting code to BS4 | #porting-code-to-bs4                                                | inte   |
| 4 | 这篇文档当然还有中文版.        | https://www.crummy.com/software/BeautifulSoup/bs4/doc.zh/           | docume |

## 7. Feature 5 – Code Example Extractor

Extracts all Python code examples, counts their lines of code, identifies specific BS4 API calls, and saves them to `data/processed/code_examples.csv`.

```
In [7]: code_df = extractor.extract_code_examples(soup)
        extractor.save_code_examples(code_df)
        print(f"Feature 5 completed: Extracted {len(code_df)} code blocks.")
        code_df.head(5)
```

Feature 5 completed: Extracted 220 code blocks.

| Out[7]: | example_id | section_title | code_text                                                                                                  | line_count | contai |
|---------|------------|---------------|------------------------------------------------------------------------------------------------------------|------------|--------|
| 0       | ex_1       | Quick Start ¶ | html_doc = """<html><head><title>The<br>Dormouse's story</title><br></head>\n<body>\n<p class="title"><... | 12         |        |
| 1       | ex_2       | Quick Start ¶ | from bs4 import BeautifulSoup\nsoup =<br>BeautifulSoup(html_doc,<br>'html.parser')\n\nprint(soup.prett...  | 36         |        |
| 2       | ex_3       | Quick Start ¶ | soup.title\n# <title>The Dormouse's<br>story</title>\n\nsoup.title.name\n#<br>u'title'\n\nsoup.title.s...  | 28         |        |
| 3       | ex_4       | Quick Start ¶ | for link in<br>soup.find_all('a'):\n    print(link.get('href'))\n#<br>http://example.com/elsie\n# http...  | 5          |        |
| 4       | ex_5       | Quick Start ¶ | print(soup.get_text())\n# The Dormouse's<br>story\n#\n# The Dormouse's story\n#\n#\nOnce upon a time...    | 12         |        |

## 8. Feature 6 – Documentation Analytics

Using Pandas and NumPy, answers all 8 required analytics questions and 2 additional questions.

```
In [8]: from src import analyzer

analytics_results = analyzer.analyze()
```

=====

## FEATURE 6: DOCUMENTATION ANALYTICS

=====

Q1: How many sections are in the documentation?

-> 113 sections

Q2: Which section has the highest word count?

-> "Searching the tree ¶" (4711 words)

Q3: Which section contains the most code examples?

-> "Searching the tree ¶" (208 code blocks)

Q4: Which section contains the most links?

-> "Table of Contents" (127 links)

Q5: What are the top 10 most frequent technical keywords?

1. soup: 926 times
2. tag: 746 times
3. class: 511 times
4. html: 420 times
5. example: 419 times
6. string: 387 times
7. com: 383 times
8. href: 378 times
9. http: 363 times
10. beautiful: 351 times

Q6: How many internal and external links exist?

-> Page-internal anchor links: 470

-> Documentation links (domain-internal): 15

-> External domain links: 17

-> Image links: 0

-> Empty or invalid links: 2

\* Note: If documentation links are grouped as internal, total internal is 48

5.

Q7: How many code examples use find\_all()?

-> 41 examples

Q8: How many code examples use get\_text()?

-> 4 examples

-----

## ADDITIONAL ANALYTICAL QUESTIONS

-----

Q9 (Additional): Average lines of code examples & section with the longest example?

-> Average lines per code example: 6.34 lines

-> Longest code example is in section "Parsing only part of a document ¶" (37 lines)

Q10 (Additional): Statistics by heading level (h1, h2, h3):

| section_level | word_count  |       | code_block_count |     | link_count |     |
|---------------|-------------|-------|------------------|-----|------------|-----|
|               | mean        | sum   | mean             | sum | mean       | sum |
| h1            | 1011.210526 | 19213 | 40.947368        | 778 | 18.631579  | 354 |
| h2            | 295.711538  | 15377 | 12.038462        | 626 | 4.846154   | 252 |
| h3            | 153.833333  | 6461  | 8.261905         | 347 | 4.714286   | 198 |

=====

## 9. Feature 7 – Data Visualization

Generates 4 required charts using Matplotlib and saves them to `output/charts/` :

- Bar chart: Top 10 sections by word count
- Bar chart: Number of code examples by section
- Pie chart: Link type distribution
- Histogram: Code example line count distribution

```
In [9]: from src import visualizer

chart_paths = visualizer.visualize()

# Display charts inline
import matplotlib.pyplot as plt
from IPython.display import Image, display

for chart_path in chart_paths:
    print(f"\n--- {chart_path.name} ---")
    display(Image(filename=str(chart_path)))
```

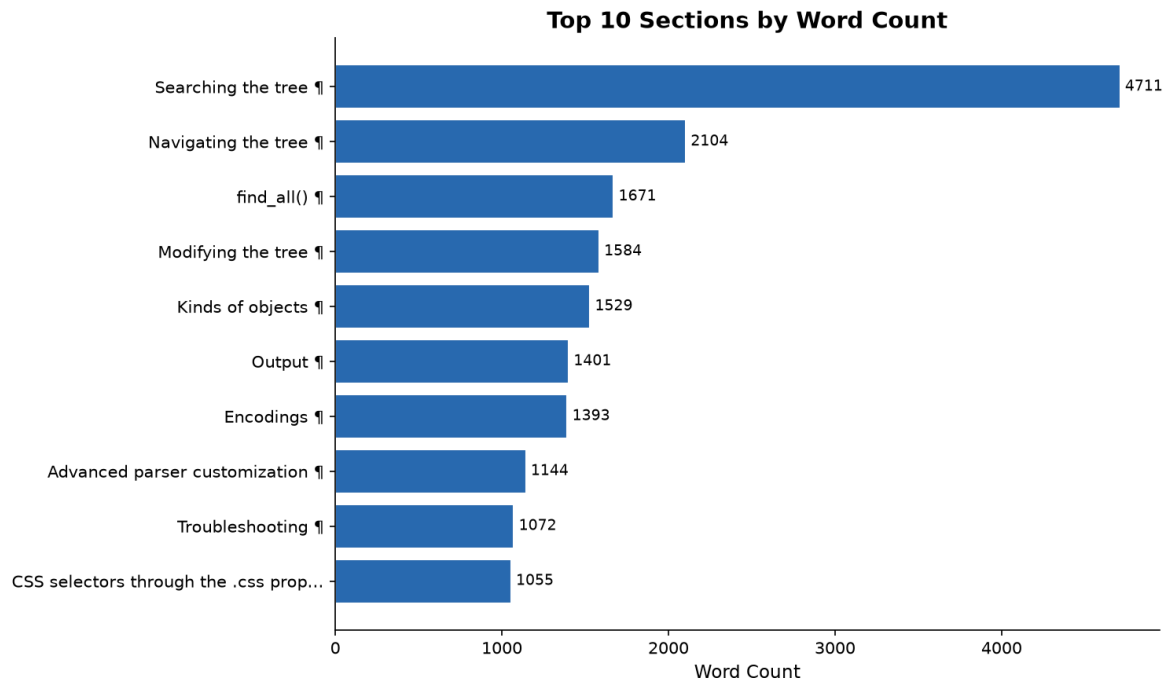
```
=====
                        FEATURE 7: DATA VISUALIZATION
=====

[1/4] Bar chart - Top 10 sections by word count -> output\charts\chart_top_sections_word_count.png
[2/4] Bar chart - Code examples by section      -> output\charts\chart_code_examples_by_section.png
[3/4] Pie chart - Link type distribution         -> output\charts\chart_link_type_distribution.png
[4/4] Histogram - Code line count distribution  -> output\charts\chart_code_line_count_histogram.png

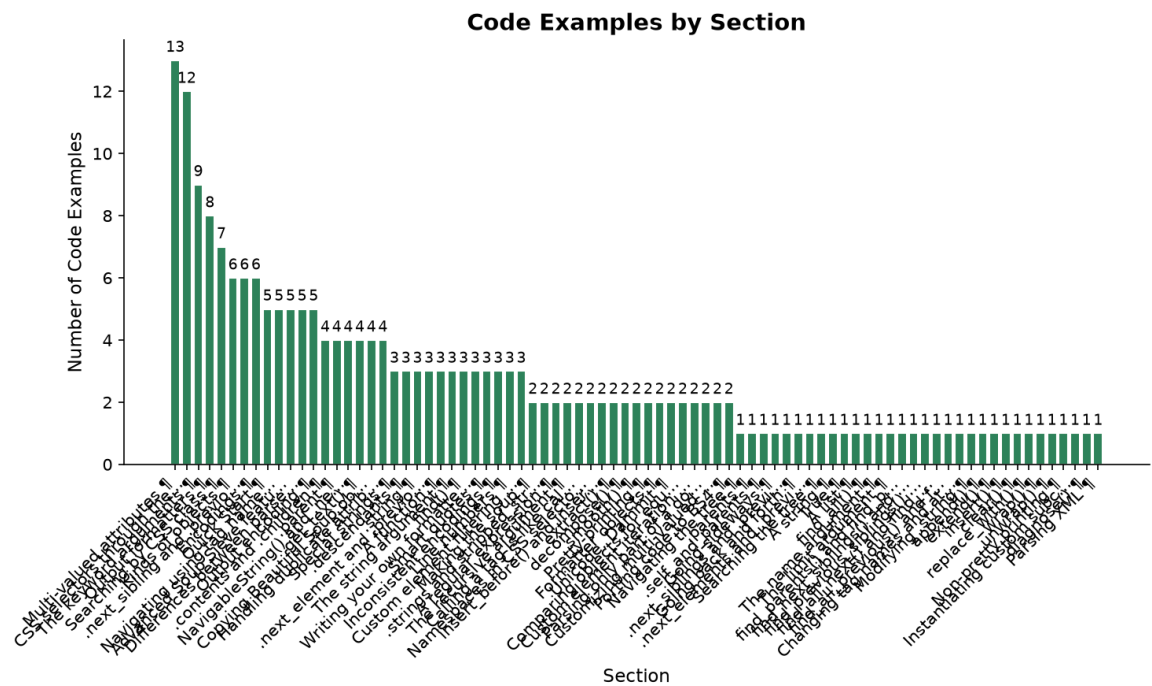
All 4 charts saved to: output\charts
=====

--- chart_top_sections_word_count.png ---
```



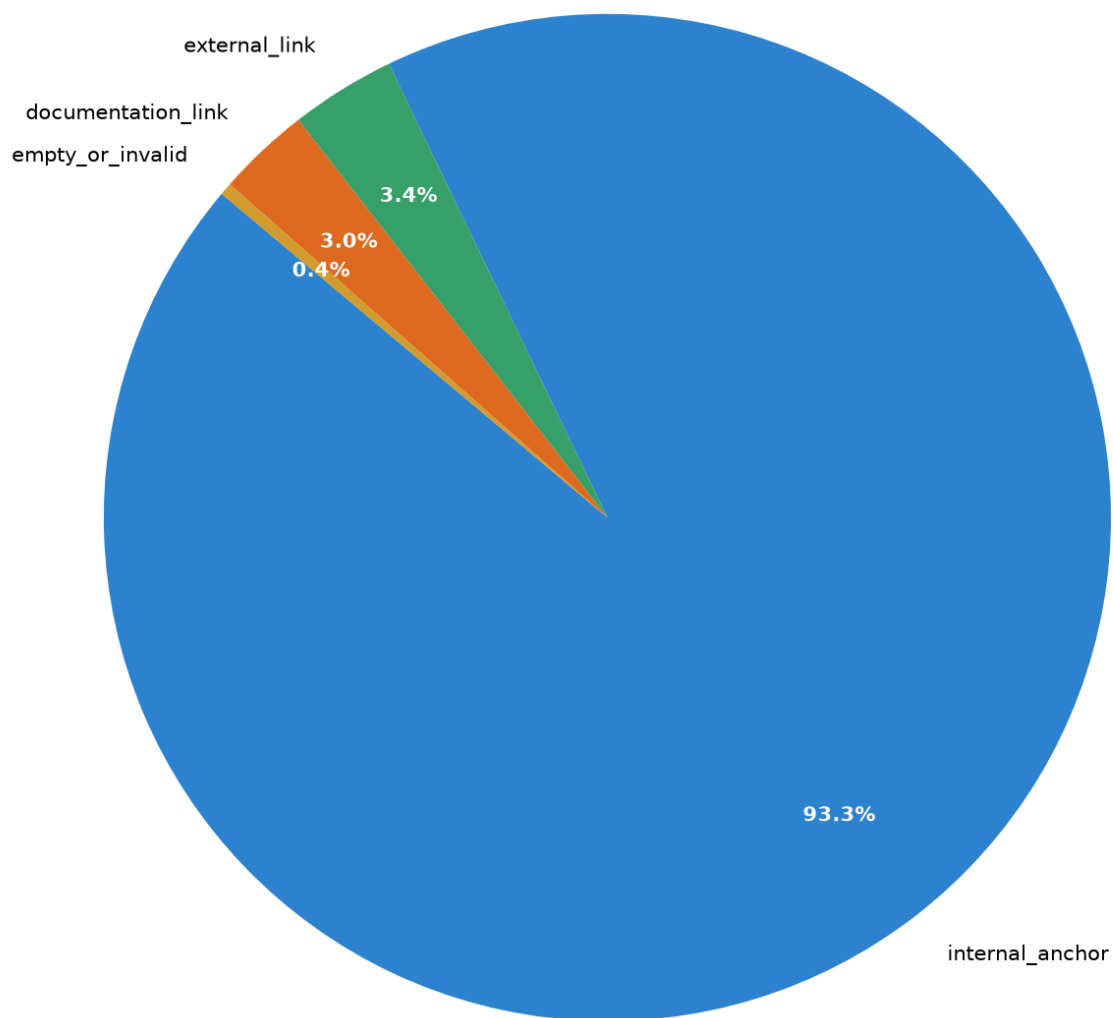


--- chart\_code\_examples\_by\_section.png ---



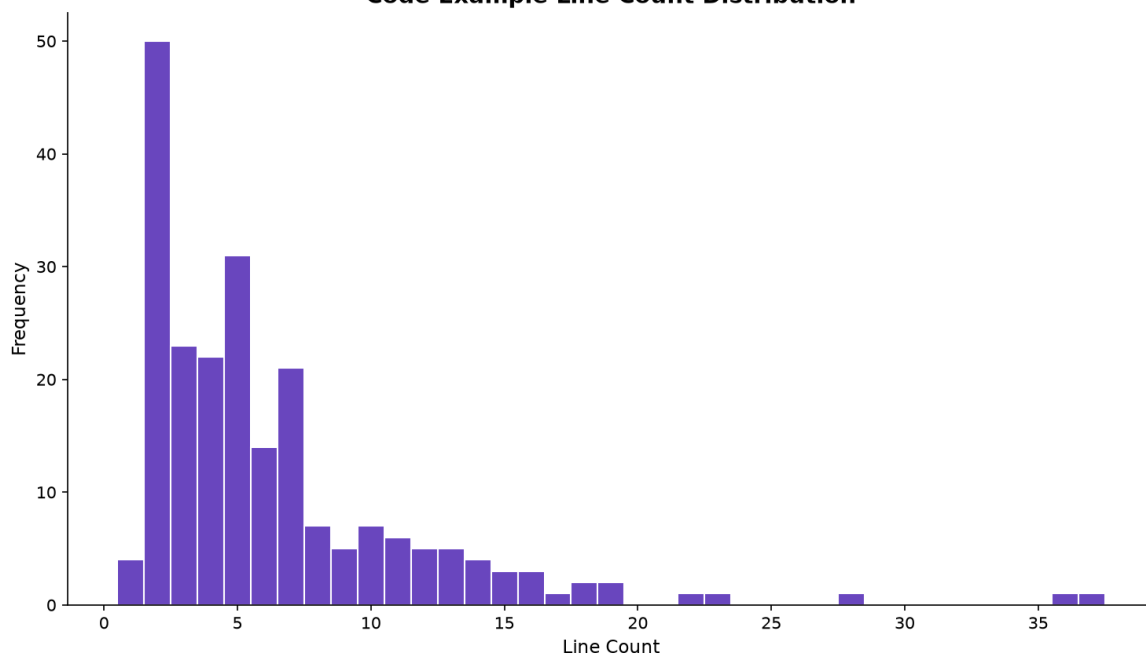
```
--- chart_link_type_distribution.png ---
```

**Link Type Distribution**



--- chart\_code\_line\_count\_histogram.png ---

**Code Example Line Count Distribution**



## 10. Feature 8 – Final Report Generator

Generates a comprehensive final analytical report in Markdown and styled HTML, including dataset overview, scraping method, extracted data summary, analysis results, embedded charts, key findings, limitations, and conclusion.

```
In [10]: from src import report_generator

md_path, html_path = report_generator.generate_report(
    results=analytics_results, chart_paths=chart_paths
)
print(f"\nReport generated:\n  Markdown: {md_path}\n  HTML:      {html_path}")

=====
FEATURE 8: FINAL REPORT GENERATOR
=====

Markdown report saved: output\final_report.md
HTML report saved:      output\final_report.html
=====

Report generated:
  Markdown: output\final_report.md
  HTML:      output\final_report.html
```

## Submission Checklist

Verify that the following project components are present before finalizing:

- ☒ src/collector.py (Feature 1)
- ☒ src/parser.py (Feature 2)
- ☒ src/extractor.py (Feature 3, 4, 5)
- ☒ src/analyzer.py (Feature 6)
- ☒ src/visualizer.py (Feature 7)
- ☒ src/report\_generator.py (Feature 8)
- ☒ src/main.py (Orchestrator — Features 1-8)
- ☒ data/raw/beautifulsoup\_doc.html (Raw HTML)
- ☒ data/processed/sections.csv , links.csv , code\_examples.csv  
(Extracted CSVs)
- ☒ output/charts/ (4 visualization charts)
- ☒ output/final\_report.md (Markdown report)
- ☒ output/final\_report.html (Styled HTML report)
- ☒ notebooks/FinalProject\_BeautifulSoup\_Analysis.ipynb (This notebook)